

Searchable Documents Project

Find in Images

Making Scanned PDFs and Image-Based Documents
Searchable with In-Browser OCR and DOM Text Injection

Suhani Pandey

A Chrome Extension (Manifest V3)

June 22, 2026

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Key Idea	3
1.3	Contributions	3
1.4	Report Structure	4
2	Functional Requirements (User Stories)	4
2.1	User Stories	4
2.2	Functional Requirements	5
2.3	Non-Functional Requirements	5
3	Background and Related Work	6
3.1	The Searchable-PDF Problem	6
3.2	Optical Character Recognition	6
3.3	In-Browser PDF Rendering with PDF.js	6
3.4	The Chrome Extension Platform (Manifest V3)	6
3.5	Client-Side Storage	7
3.6	Positioning Relative to Existing Approaches	7
4	Technology Stack and Integration	7
4.1	The Stack at a Glance	7
4.2	How Each Technology Is Used	8
4.3	Coordination Between Technologies	8
5	System Design and Architecture	9
5.1	Overview	9
5.2	Components	9
5.3	The Coordinate-Mapping Problem	11
5.4	Key Design Decisions	11
6	Development Process: Phased Implementation	12
6.1	Planning: The Initial Estimate	12
6.2	Execution: How the Build Actually Unfolded	12
6.3	Plan versus Reality	14
7	Implementation	14
7.1	Extension Structure and PDF Interception	14
7.2	Lazy Rendering Pipeline	15
7.3	Text Detection	15
7.4	Native Text-Layer Injection	15
7.5	OCR Rendering and Preprocessing	16
7.6	OCR Engine Orchestration	16
7.7	Post-Processing for Findability	17
7.8	Normalization, Caching, and De-duplication	17
7.9	Search and the Custom Find Bar	17
8	Results and Evaluation	18
8.1	Functional Outcomes	18
8.2	Architectural Behavior	18
8.3	Component Complexity	19

8.4 Discussion	19
9 Limitations and Future Work	19
9.1 Limitations	19
9.2 Future Work	20
10 Conclusion	20
References	21

Abstract

Scanned PDFs and image-based documents are ubiquitous, yet a large fraction of them carry no machine-readable text layer, so the browser’s familiar “find on page” (Ctrl+F) returns nothing. This report presents FIND IN IMAGES, a Chrome extension (Manifest V3) that closes this gap entirely on the client. Rather than building a bespoke search engine, FIND IN IMAGES *makes the document itself searchable*: it intercepts PDF navigations, renders pages with PDF.js, detects which pages paint raster images or lack extractable text, runs optical character recognition (OCR) on those pages with Tesseract.js in Web Workers, and injects an invisible, precisely-aligned text layer over the rendered canvas so that search operates on real DOM text.

The central engineering problem is geometric: OCR and PDF text live in different coordinate spaces (PDF points, the PDF.js viewport, the device-pixel canvas, and the CSS-pixel DOM overlay), and the injected glyphs must line up with the visible page across zoom levels and high-DPI displays. We describe the affine transformation pipeline that resolves this, a measure-and-scale technique that fits each invisible word to its target box, a high-resolution OCR rendering path with contrast-stretch and unsharp-mask preprocessing, resolution-independent caching in IndexedDB, and an OCR post-processing stage that emits search-only correction variants to improve hit-rate. We also discuss a custom find bar that replaces the native one to avoid rendering artifacts. The result is a fully offline, client-side system that turns static documents into searchable, web-native experiences.

1 Introduction

1.1 Motivation

Documents on the web fall broadly into two categories. *Text-based* PDFs embed their characters as selectable, searchable glyphs and work seamlessly with the browser’s built-in find feature. *Image-based* documents—scanned pages, photographs of text, screenshots, and PDFs whose text has been flattened into raster images—contain no machine-readable text at all. To the browser they are pictures. Pressing Ctrl+F on such a document finds nothing, even though the text is plainly visible to a human reader.

This is a pervasive friction point. A student searching a scanned textbook, a researcher scanning archival material, or anyone working with a faxed or photographed form repeatedly hits the same wall: the content is *there*, but not *findable*. The conventional remedy—uploading the file to a server-side OCR service to produce a new, searchable PDF—is slow, raises privacy concerns, and breaks the natural in-browser reading flow.

1.2 Key Idea

FIND IN IMAGES takes a deliberately different stance. Instead of *reinventing* search—building an index, a query parser, and highlight logic—it *transforms the document so the browser can search it natively*. Every page is rendered, any text baked into images is recovered with OCR, and the recovered text is injected back into the page as invisible, exactly-positioned DOM elements. From the browser’s perspective the document now *is* text, and the entire mature machinery of in-page search applies for free.

This inversion is the conceptual core of the project: *the cheapest way to make something searchable is to make it real text*, not to intercept and re-implement the act of searching.

1.3 Contributions

This report documents the design and implementation of FIND IN IMAGES. Its principal contributions are:

- A fully **client-side, offline** pipeline that makes scanned PDFs searchable in the browser, with no server round-trip and no data leaving the user’s machine.
- A **coordinate-mapping engine** that aligns OCR output and native PDF text across four distinct coordinate spaces, remaining correct under zoom, page rotation, and high-DPI rendering (Section 5).
- A **performance architecture**—lazy, viewport-driven rendering plus idle-time background indexing on Web Workers, with resolution-independent caching in IndexedDB—that keeps the interface responsive on large documents (Section 7).
- An **OCR accuracy layer**: high-resolution rendering with targeted image preprocessing, plus post-processing that emits search-only correction variants to raise the find hit-rate (Section 7).

1.4 Report Structure

Section 2 states the system’s functional requirements as user stories. Section 3 surveys the relevant background and prior approaches. Section 4 catalogues the technologies used and how they coordinate. Section 5 presents the system architecture and the coordinate-mapping problem at its heart. Section 6 recounts the phased development of the project and the reasoning behind each phase, and Section 7 details the implementation of each pipeline stage. Section 8 evaluates the system qualitatively, Section 9 discusses limitations and future work, and Section 10 concludes.

2 Functional Requirements (User Stories)

Before the architecture, it is worth stating plainly what the system must do for its users. FIND IN IMAGES has essentially one kind of user—anyone viewing a PDF in Chrome—and one overriding need: to find and reuse text that is visible on the page, whether or not the document stores it as real text. The requirements below are written first as user stories, then distilled into functional and non-functional requirements that the rest of the report can be traced against.

2.1 User Stories

- **Find anything visible.** *As a reader of a scanned PDF, I want Ctrl+F to find words that appear only inside images, so that no visible text is unsearchable.*
- **Mixed pages.** *As a reader of a page that mixes real text and text baked into images, I want both to be searchable, without a word being matched twice.*
- **Select and copy.** *As a user, I want to select and copy the recovered text, so that I can quote or reuse it elsewhere.*
- **Navigate matches.** *As a user, I want to step through matches with $\uparrow$ / $\downarrow$ and see each one clearly highlighted, even on colourful pages.*
- **Choose the language.** *As a reader of non-English documents, I want to pick the OCR language (English, Danish, or both).*
- **Large documents stay usable.** *As a user opening a large PDF, I want it to open immediately and stay responsive while it becomes searchable in the background.*
- **Instant on revisit.** *As a returning user, I want a document I have already opened to be searchable instantly, without re-processing.*

- **Private and offline.** *As a privacy-conscious user, I want everything to happen on my machine, with no upload and no network dependency.*
- **Leave my file alone.** *As a user, I want the original PDF displayed unchanged—never rewritten, re-saved, or reformatted.*

2.2 Functional Requirements

Table 1 restates the stories as testable functional requirements and traces each to the development phase that delivered it (Section 6).

Table 1. Functional requirements stated as testable conditions, each traced to the development phase that delivered it (Section 6).

ID	Requirement	Delivered in
FR-1	Make text inside images findable via the browser’s native search	Phases 3, 11
FR-2	Keep native PDF text searchable	Phase 3
FR-3	On mixed pages, expose both native and image text, de-duplicated	Phase 3
FR-4	Allow the recovered text to be selected and copied	Phase 10
FR-5	Provide a find bar with ↑ / ↓ navigation and clear highlights	Phase 10
FR-6	Support multiple OCR languages (English, Danish)	Phase 8
FR-7	Align text correctly on rotated pages	Phase 7
FR-8	Open large PDFs immediately and index them in the background	Phase 9
FR-9	Cache results so revisited documents are searchable instantly	Phase 4

2.3 Non-Functional Requirements

Table 2. Non-functional requirements — system-wide qualities that apply regardless of which page or document is being processed — and the design decisions or phases that realise each one.

ID	Requirement	Realised by
NFR-1	Non-destructive: the original PDF is never modified — read-only render plus overlay	Design principle (§5)
NFR-2	Offline & private: no network at runtime; bundled OCR engine and models	Phase 5
NFR-3	Responsive: OCR runs off the main thread; the UI never blocks	Phase 9
NFR-4	Accurate: image-text recovery as close to “fully findable” as pre-processing allows	Phases 6, 11
NFR-5	Resolution-independent: text stays aligned across zoom and high-DPI displays	§5
NFR-6	Maintainable cache: a format version invalidates stale results when the pipeline changes	Phases 4, 11

3 Background and Related Work

3.1 The Searchable-PDF Problem

The PDF format [1] can represent text either as encoded glyphs with positioning operators or as raster images. Only the former is extractable. A “scanned” PDF is typically a sequence of full-page images with no text operators, so any tool relying on the document’s text stream—including the browser’s find feature—sees an empty page. Making such a document searchable requires *recovering* the text through OCR and re-associating it with the page.

3.2 Optical Character Recognition

OCR converts images of text into character sequences. Tesseract, originally developed at HP and later open-sourced by Google, is among the most widely used engines; modern versions use an LSTM-based recognizer and report results as a hierarchy of blocks, paragraphs, lines, and words, each with a bounding box and a confidence score [2]. FIND IN IMAGES uses Tesseract.js [3], a WebAssembly port that runs entirely in the browser, which is what makes a fully client-side, offline pipeline possible.

OCR quality is sensitive to input image quality. Low contrast, blur, and low resolution all degrade recognition, which motivates the preprocessing stage described in Section 7. Classic techniques such as Otsu’s thresholding [4] binarize an image by analyzing its gray-level histogram; FIND IN IMAGES performs a related percentile-based contrast stretch before handing the image to Tesseract’s own binarizer.

3.3 In-Browser PDF Rendering with PDF.js

PDF.js [5] is Mozilla’s standards-based PDF renderer written in JavaScript. It parses a PDF, exposes each page’s content, and rasterizes pages to an HTML5 `<canvas>`. Crucially it also provides two capabilities this project depends on: `getTextContent()`, which returns positioned text items for pages that *do* have a text layer, and `getOperatorList()`, which exposes the low-level drawing operators and lets us detect whether a page paints raster images. PDF.js also defines a *viewport* abstraction that maps the PDF’s coordinate system (points, bottom-left origin) into device pixels, which is the first link in our coordinate chain.

3.4 The Chrome Extension Platform (Manifest V3)

Manifest V3 [6] is the current Chrome extension architecture. The components relevant here are:

- **Service worker** — an event-driven background script (replacing persistent background pages) used here to register network rules on install.
- **declarativeNetRequest** [7] — a declarative API for redirecting or blocking requests without intercepting their contents, used to reroute PDF navigations to the custom viewer.
- **Content scripts** — scripts injected into web pages, reserved here for future handling of image text on ordinary pages.
- **Web-accessible resources** and a **content security policy** permitting `wasm-unsafe-eval`, required so the bundled WebAssembly OCR core can execute.

3.5 Client-Side Storage

Because OCR is computationally expensive, recovered text is cached in IndexedDB [8], a transactional, asynchronous, origin-scoped key–value store suited to structured data. Caching is what turns a slow first read into an instant subsequent one.

3.6 Positioning Relative to Existing Approaches

Server-side services (e.g. commercial PDF tools and the open-source `ocrmypdf`) produce a new searchable PDF but require an upload and a round-trip, and they replace the user’s in-browser reading flow. Browser-native PDF viewers render scanned PDFs but cannot search them. FIND IN IMAGES occupies the gap between these: it keeps the live, in-browser experience while adding searchability locally and privately, and it reuses the browser’s own search rather than shipping a new one. The trade-off is that recognition runs on the client’s CPU; the architecture in Section 5 is largely a response to making that trade-off acceptable.

4 Technology Stack and Integration

Every part of FIND IN IMAGES runs on the client, inside the browser. The system is assembled from a small set of web technologies, each chosen for one job, and the engineering value is largely in how they are *coordinated*. This section catalogues the stack, explains how each piece is used, and then shows how they fit together as a single pipeline.

4.1 The Stack at a Glance

Table 3 lists each technology, the single job it performs, and the file where it is wired into the system; the subsections that follow expand on how each one is used.

Table 3. Technologies used in FIND IN IMAGES, the single role each plays, and the source file where it is wired into the system.

Technology	Role in the project	Where
Chrome Extension (MV3)	Packaging, lifecycle, and the permission/CSP model that lets WASM OCR run	<code>manifest.json</code>
Service worker + declarative NetRequest	Intercepts PDF navigations and redirects them to the custom viewer	<code>service-worker.js</code>
PDF.js (<code>pdfjs-dist</code>)	Parses and renders pages; supplies native text and image-operator info	<code>viewer.js</code>
HTML5 Canvas API	Rasterization target — both the on-screen page and the offscreen OCR image	<code>viewer.js</code>
Tesseract.js (WASM)	Optical character recognition on image pages	<code>viewer.js</code>
Web Workers + scheduler	Run OCR off the main thread, several pages at once	<code>viewer.js</code>
IndexedDB	Persistent, resolution-independent cache of OCR results	<code>cache.js</code>
<code>localStorage</code>	Remembers the chosen OCR language	<code>viewer.js</code>
DOM + <code>Range</code> / <code>IntersectionObserver</code> / <code>requestIdleCallback</code>	Text-layer overlay, find-highlight geometry, lazy render, idle indexing	<code>viewer.js</code> , <code>text-layer.js</code>

4.2 How Each Technology Is Used

Chrome Extension (Manifest V3).

The `manifest.json` declares the `storage` and `declarativeNetRequest` permissions, exposes the viewer and bundled libraries as `web_accessible_resources`, and sets a content security policy of `script-src 'self' 'wasm-unsafe-eval'` — the last clause is what permits Tesseract’s WebAssembly core to execute inside the extension.

Service worker & `declarativeNetRequest`.

On install the service worker registers one *dynamic* redirect rule whose regex matches `.pdf` URLs and rewrites them to `viewer.html?url=\0`. The rule is dynamic (not static) because it must embed the extension’s runtime URL, which is unknown until install time.

PDF.js.

`getDocument({url})` loads the file; per page, `getViewport({scale})` sizes it and `render()` draws it to a canvas. `getTextContent()` yields native text items, and `getOperatorList()` (checked against `pdfjsLib.OPS` image operators) reveals whether a page paints raster images. The `viewport.transform` matrix anchors the coordinate mapping of Section 5.

Canvas API.

Each page is rendered twice: an on-screen canvas whose backing store is scaled by the device pixel ratio for sharpness, and a separate offscreen canvas targeting ~ 3000 px wide for OCR. `getImageData` / `putImageData` give the preprocessing stage pixel-level access.

Tesseract.js.

Loaded as a UMD script that exposes `window.Tesseract`. A `createScheduler()` fronts a pool of `createWorker()` LSTM workers configured with `PSM.SPARSE_TEXT`; an `addJob("recognize")` call (with `blocks:true`) returns the block $\rightarrow$ word hierarchy with bounding boxes. The WASM cores and `eng/dan` language data are bundled locally, so OCR is fully offline. Three language modes are supported: English (`eng`), Danish (`dan`), and combined English+Danish (`eng+dan`).

Web Workers & scheduler.

OCR is CPU-bound, so it never runs on the main thread. The scheduler distributes pages across $\min(4, \text{cores}-1)$ workers, keeping the UI responsive while several pages recognize in parallel.

IndexedDB.

`cache.js` stores each page’s recognized words under a `${docId}::${pageNum}` key, with coordinates normalized to the unit square and a `FORMAT_VER` stamp that invalidates results when the pipeline changes.

localStorage & DOM APIs.

The OCR language is remembered in `localStorage`. The recovered text becomes invisible `<span>s`; `Range.getClientRects()` provides exact geometry for find highlights; `IntersectionObserver` triggers lazy rendering; and `requestIdleCallback` paces background indexing.

4.3 Coordination Between Technologies

`viewer.js` is the orchestrator: it owns the page lifecycle and brokers every hand-off between the libraries. Figure 1 traces the data that crosses each boundary. A PDF request is rerouted by the service worker; PDF.js rasterizes pages to canvas and reports their text and image operators; image pages are sent (as a high-resolution canvas) to the Tesseract worker pool, which returns words and boxes; those words are normalized and persisted to IndexedDB; and

finally the orchestrator emits the invisible DOM text layer that the find bar searches. Each technology only ever talks to `viewer.js`, which keeps the coupling between them loose and the data contracts explicit.

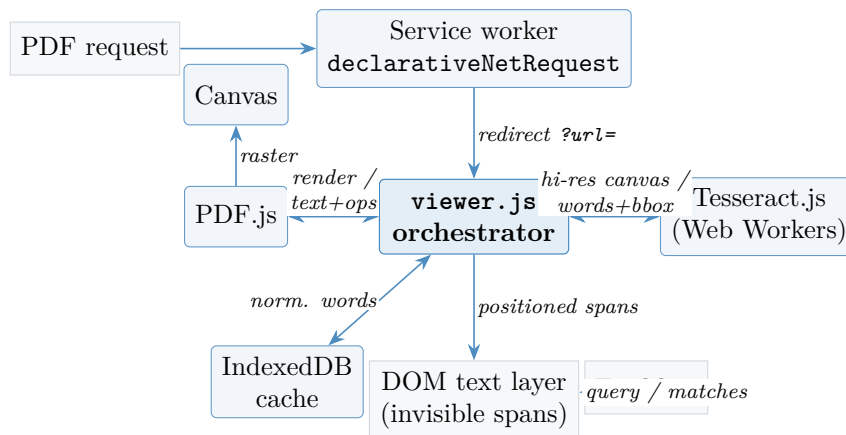


Figure 1. Integration view. `viewer.js` acts as the sole orchestrator — every library and storage layer communicates only with it, so inter-component coupling stays low. Arrow labels name the data crossing each boundary; double-headed arrows indicate request–response pairs.

5 System Design and Architecture

5.1 Overview

FIND IN IMAGES is structured as a *pipeline* that converts an incoming PDF request into searchable DOM text. A PDF navigation is intercepted and redirected to a custom viewer; the viewer renders each page, checks whether it paints raster images or lacks extractable text, recovers image text with OCR where needed, normalizes coordinates, and injects an invisible text layer that the browser’s find feature can search. Figure 2 shows the end-to-end flow.

5.2 Components

Request interception

A service worker registers a dynamic `declarativeNetRequest` rule on install that matches URLs ending in `.pdf` and redirects them to `viewer.html`, preserving the original URL as a query parameter.

Custom viewer

A self-contained HTML/JS page that owns rendering, OCR orchestration, the text layer, and the find UI.

Rendering

PDF.js rasterizes each page to a canvas, lazily and at the device pixel ratio for crisp output.

Text detection

For each page the viewer asks PDF.js for its text content and operator list to decide whether OCR is required.

OCR

A pool of Tesseract.js Web Workers recognizes text on image pages, returning words with bounding boxes.

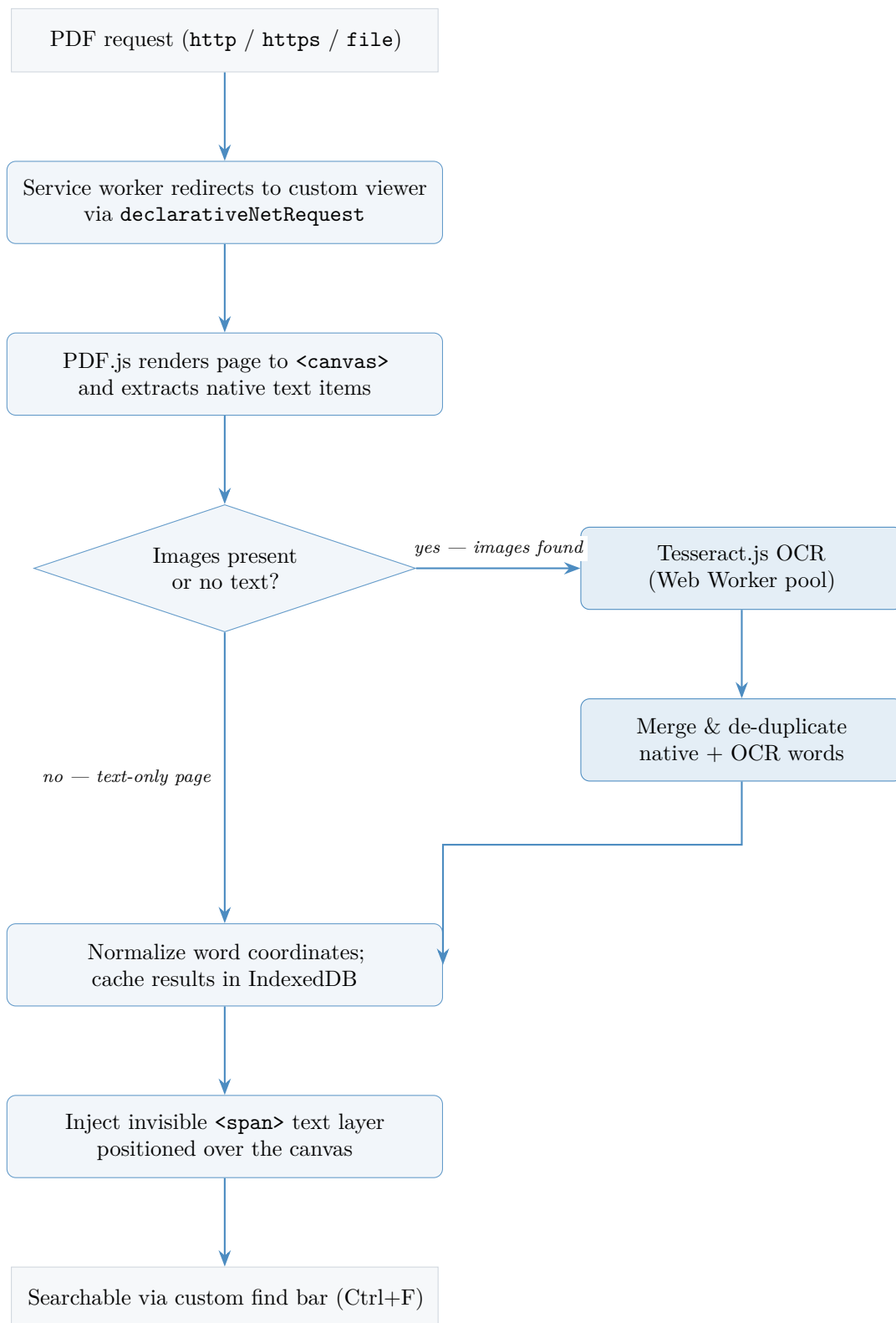


Figure 2. End-to-end processing pipeline. Each page is rendered and its native text extracted. If the page contains raster image operators *or* has no extractable text (right branch), Tesseract.js runs OCR on a high-resolution offscreen canvas and the results are merged with any native text, dropping duplicates. Text-only pages skip OCR and pass straight to normalization (left path). Both paths converge at coordinate normalization before the invisible text layer is injected over the canvas.

Normalization & caching

Word coordinates are normalized to the unit square and cached in IndexedDB so results are reusable across zoom levels and sessions.

Text-layer injection

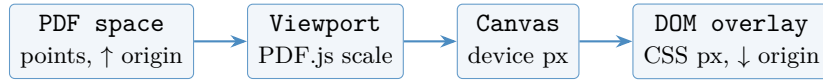
Invisible, individually-scaled `<span>` elements are positioned over the canvas.

Search

A custom find bar searches the injected text and draws highlight overlays.

5.3 The Coordinate-Mapping Problem

The hardest part of the system is geometric. Text originates in up to four distinct coordinate spaces, and the injected overlay must align with the rendered page in all of them simultaneously:



PDF space uses typographic points with a bottom-left origin; the PDF.js viewport applies a scale (and any page rotation); the canvas backing store is further scaled by the device pixel ratio for high-DPI sharpness; and the DOM overlay uses CSS pixels with a top-left origin. A glyph recovered or extracted in one space must be drawn at the corresponding location in the overlay, or search highlights and text selection drift away from the visible characters.

For *native* PDF text, PDF.js supplies, per item, an affine transform M_{item} and the viewport's transform M_{vp} (which already folds in page rotation). The glyph's full matrix in device space is their product

$$M = M_{\text{vp}} M_{\text{item}}, \quad M = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix}, \quad (1)$$

from which the baseline angle and device-space font height follow directly:

$$\theta = \text{atan2}(b, a), \quad h = \sqrt{c^2 + d^2}. \quad (2)$$

Because rotation falls out of the matrix in Eq. (2), rotated pages and rotated text are handled without special cases. The span's top-left corner is obtained by stepping from the baseline origin (e, f) up by the ascent $h_a \approx 0.8h$ along the text direction.

For *OCR* text, Tesseract reports pixel bounding boxes in the high-resolution OCR canvas. These are normalized to the unit square by dividing by the OCR canvas dimensions,

$$\tilde{x} = \frac{x}{W_{\text{ocr}}}, \quad \tilde{y} = \frac{y}{H_{\text{ocr}}}, \quad (3)$$

so the cached result is independent of the resolution at which OCR happened. At draw time they are denormalized into the text layer's CSS-pixel space using the *viewport* dimensions—not the DPR-scaled canvas dimensions—so the invisible text and the visible page share one coordinate system.

5.4 Key Design Decisions

Four decisions shape the rest of the system and are justified where they appear in Section 7:

1. **Never modify the original PDF.** The system is a strictly read-only render plus an overlay: the source file's bytes and format are left untouched, and all recovered text lives in a separate, invisible DOM layer. This constraint, fixed from the outset, rules out any

rewrite-the-PDF approach and is the reason searchability is added *around* the document rather than *into* it.

2. **Inject real text instead of intercepting search.** This reuses the browser’s indexing, matching, and (optionally) highlighting, and keeps recovered text selectable and copyable.
3. **Separate the OCR image from the display image.** Display fidelity and OCR legibility have different requirements, so the page is rendered twice: once for the screen at the device pixel ratio, and once offscreen at high resolution purely for recognition.
4. **Lazy rendering with background indexing.** Pages render as they approach the viewport for responsiveness, while a separate idle-time pass guarantees that off-screen pages still become searchable.

6 Development Process: Phased Implementation

FIND IN IMAGES was built incrementally, one phase at a time, with each phase shipped and tested before the next began. Two fixed principles governed the whole effort: the original PDF is **never modified**—the system is strictly a read-only render plus an overlay—and the headline capability, **finding text inside images**, had to be as close to “100% findable” as possible. This section records how the work was planned, how it actually unfolded, and the reasoning at each step.

6.1 Planning: The Initial Estimate

Before writing code, the work was broken into seven phases with rough effort estimates (Table 4). Two were flagged up front as the likely time-sinks: the *invisible text-layer overlay* (the coordinate-alignment problem) and *intercepting Chrome’s built-in PDF viewer*, which is sandboxed and cannot simply be scripted.

Table 4. Initial seven-phase plan drawn up before implementation began. Phases 5 and 6 were flagged as the expected time-sinks; Phase 6 was ultimately pulled forward and solved first.

#	Planned phase	Note
1	Extension scaffolding (MV3)	Manifest, service worker, content-script wiring
2	PDF.js rendering pipeline	Pages to canvas; worker setup in an extension context
3	Tesseract.js OCR	Plumbing OCR into the async page pipeline
4	Per-page text detection	<code>getTextContent()</code> check — quick
5	Invisible text-layer overlay	<i>Hardest</i> : PDF↔canvas↔DOM coordinate alignment
6	Intercept Chrome’s PDF viewer	<i>Hidden complexity</i> : sandboxed viewer; must redirect
7	Polish, edge cases, testing	—

6.2 Execution: How the Build Actually Unfolded

Execution diverged from the plan in two instructive ways. First, viewer interception (planned as Phase 6) turned out to be a *prerequisite* for everything else—you cannot render or overlay anything until you control the page—so it was solved first, folded into Phase 1. Second, “OCR accuracy” was not a single step but the dominant theme of the back half of the project: it expanded into several phases as the README roadmap and real test feedback exposed new failure modes. The phases below are the build as it actually happened.

Foundation.

Phase 1 — Extension skeleton & PDF interception.

Built the MV3 manifest, service worker, content script, and an empty viewer, then made the service worker redirect `.pdf` navigations to that viewer. *Why first:* Chrome opens PDFs in its own sandboxed viewer that scripts cannot reach, so gaining control of the page is the true starting point—and it establishes the read-only-overlay architecture. *Revealed:* a viewer that shows nothing yet → Phase 2.

Phase 2 — PDF.js rendering pipeline.

Bundled PDF.js locally (to satisfy the MV3 CSP) and rendered each page to a canvas, switching to a *dynamic* redirect rule so the original URL survives as `?url=`. *Friction:* `file://` security origins and the `getDocument` URL parameter both needed care. *Revealed:* pages display but nothing is searchable → Phase 3.

Phase 3 — Text detection, OCR & the invisible overlay.

The heart of the project: detect native text with `getTextContent()`, OCR image pages with Tesseract.js in a worker, and inject invisible spans over the canvas. *Key pivot:* the first design was *either/or* (use native text, else OCR), but a page containing the same word both as native text *and* inside an image matched only the native copy—image text was lost. The design changed to *both:* always inject native text, then OCR images and merge only the non-duplicate words (text-aware de-duplication). *Revealed:* OCR re-ran on every load → Phase 4.

Phase 4 — IndexedDB caching.

Cache each page's words once, keyed per document, with coordinates normalized to $[0, 1]$ so the cache survives any zoom level. *Why:* OCR is the expensive step, so it should be a one-time cost. *Revealed:* language data still implied a CDN → Phase 5.

Phase 5 — Fully offline.

Bundled the LSTM language model and pointed Tesseract's `langPath` at it, removing the last network dependency. *Why:* privacy, reliability, and offline use—no document data ever leaves the machine.

Refinement (working through the README roadmap and test feedback).

Phase 6 — OCR accuracy: correction variants.

Added confidence filtering, edge-noise trimming, and digit→letter correction variants (`he11o`→`hello`) injected as search-only spans. *Thought:* native find is exact-match, so rather than fuzzy-matching at query time, widen the *stored* text.

Phase 7 — Rotated pages.

Derived each glyph's angle and size from the `viewport.transform × item.transform` matrix and rotated the spans to match, so rotation is handled by the math rather than special cases (Section 5).

Phase 8 — Multi-language OCR.

Added an English/Danish selector with a per-language cache, so switching languages reprocesses independently and non-destructively.

Phase 9 — Performance on large PDFs.

Introduced lazy, viewport-driven rendering, idle-time background indexing, and *parallel* OCR via a worker scheduler. *Thought:* a single serial OCR worker was the real bottle-

neck; the roadmap’s “GPU acceleration” proved not actionable (Tesseract.js has no GPU path and already uses the SIMD WASM core), so parallelism was the available win.

Phase 10 — Readability & search UX.

Driven directly by user testing: the text layer was made selectable/copyable; a measure-and-scale fix aligned each invisible span to its exact width after selection and search landed on the wrong characters (“Where is”→“Where is the b”); and the native find bar—which flashed the invisible layer as oversized doubled text—was replaced by a custom find bar with translucent highlights, ↑ / ↓ navigation, and a distinct active-match style for colourful pages.

Phase 11 — OCR image preprocessing.

The final push on the “100% findable” goal, targeting the cases still failing: sparse-text segmentation recovered *tables*; a high-resolution (3000 px) OCR canvas recovered *small* text; and a percentile contrast stretch plus unsharp mask attacked *low-contrast and blurred* text (the cache format version was bumped to invalidate stale results). *Thought*: decouple the OCR image from the display image and push recognition as far as preprocessing allows—while accepting a hard ceiling on genuinely low-resolution source images.

6.3 Plan versus Reality

The original instinct that the *overlay* would be the hardest part was correct—coordinate alignment, the native-versus-image de-duplication, and the measure-and-scale fitting absorbed the most iteration. The estimate was wrong in its *shape*, though: “OCR” looked like one tidy phase but became the project’s centre of gravity, because the headline promise (find *any* text in *any* image) is exactly the kind of goal that surfaces a long tail of real-world failure modes—rotation, tables, multiple languages, blur, low contrast, small type. Each was its own phase, and each was reached by shipping the previous one and watching where it still fell short.

7 Implementation

This section walks through the pipeline stage by stage. Code excerpts are drawn from the extension’s source and lightly trimmed for space.

7.1 Extension Structure and PDF Interception

The extension is a Manifest V3 package. Its content security policy permits `wasm-unsafe-eval` so the bundled Tesseract WebAssembly core can run, and the OCR engine, language data, and PDF.js are declared as web-accessible resources so the viewer can load them offline—no CDN fetch occurs at runtime.

On installation, the service worker registers a single dynamic `declarativeNetRequest` rule. The extension’s runtime ID is not known until install time, so the rule is created programmatically rather than statically (Listing 1). The regular expression matches `http`, `https`, and `file` URLs ending in `.pdf`, and the `regexSubstitution` of `\0` folds the entire matched URL into the viewer’s `?url=` parameter.

```
1 await chrome.declarativeNetRequest.updateDynamicRules({
2   removeRuleIds: [1],
3   addRules: [{
4     id: 1, priority: 1,
5     action: { type: "redirect",
6       redirect: { regexSubstitution: `${viewerBase}?url=\0` } },
7     condition: {
8       regexFilter: "^(https?|file)://.*\\.pdf(\\?.*)?$",
```



```

9     resourceTypes: ["main_frame", "sub_frame"],
10   },
11 },
12 });

```

Listing 1. Dynamic `declarativeNetRequest` rule installed at extension startup: intercepts .pdf URLs and redirects them to the custom viewer, embedding the original URL as a `?url=` parameter (`service-worker.js`).

The viewer then reads the original URL from its query string. Because `URLSearchParams` percent-decodes values, the raw string is re-encoded with `encodeURIComponent` before being handed to `PDF.js`, preserving spaces and special characters in the path while leaving URL syntax intact.

7.2 Lazy Rendering Pipeline

Rendering and OCR-ing every page up front would block on large documents. Instead, the viewer immediately creates a correctly-sized placeholder `<div>` for every page—estimated from page 1’s dimensions so the scrollbar reserves the right space—and uses an `IntersectionObserver` [9] with a 300px margin to render each page just before it scrolls into view.

Each page is rendered at the device pixel ratio (capped at 2×) so text stays crisp on Retina displays: the canvas backing store is enlarged by the output scale while the canvas is *displayed* at CSS size, letting the browser downsample to a sharp image. Layout remains in CSS pixels, which keeps the text layer’s coordinate space stable regardless of display density.

7.3 Text Detection

For each page the viewer first calls `getTextContent()`. It then decides whether OCR is needed: OCR runs when the page paints raster images *or* when no text is extractable at all (a page may be a scan, or have text converted to vector outlines—both invisible to `getTextContent()`). Image detection inspects the operator list:

```

1  const IMAGE_OPS = new Set([
2    pdfjsLib.OPS.paintImageXObject,
3    pdfjsLib.OPS.paintImageMaskXObject,
4  ]);
5  async function pageHasImages(page) {
6    const opList = await page.getOperatorList();
7    return opList.fnArray.some((fn) => IMAGE_OPS.has(fn));
8  }

```

Listing 2. Image-detection function: scans the PDF.js operator list for raster paint opcodes to decide whether OCR is needed on a given page (`viewer.js`).

If image detection fails it *fails open*—assuming images are present—on the principle that an unnecessary OCR pass is cheaper than silently missing text.

7.4 Native Text-Layer Injection

When PDF text exists, it is injected directly because it is exact and free. Aligning the invisible glyphs with the canvas, however, requires care: a span’s natural rendered width will not match the PDF run’s width. The implementation uses a three-pass *measure-and-scale* technique that

triggers only one layout reflow. Each span is created and positioned using the matrix of Eq. (1)–(2); all spans are measured in a single pass; then each is scaled horizontally to exactly its target device-space width:

$$s_x = \frac{w_{\text{target}}}{w_{\text{measured}}}, \quad (4)$$

applied as a CSS `scaleX` (composed with `rotate( $\theta$ )` for rotated text). Measuring rather than estimating is what makes selection and find land on the correct characters. The matrix product itself is implemented directly:

```

1 function matMul(m1, m2) {
2   return [
3     m1[0]*m2[0] + m1[2]*m2[1],      m1[1]*m2[0] + m1[3]*m2[1],
4     m1[0]*m2[2] + m1[2]*m2[3],      m1[1]*m2[2] + m1[3]*m2[3],
5     m1[0]*m2[4] + m1[2]*m2[5] + m1[4], m1[1]*m2[4] + m1[3]*m2[5] +
      m1[5],
6   ];
7 }
```

Listing 3. Six-element 2D affine matrix product that composes the PDF.js viewport transform with each glyph’s own transform when placing native text spans (`text-layer.js`).

7.5 OCR Rendering and Preprocessing

The on-screen canvas is sized to the window and is often too low-resolution for small text inside images. OCR therefore uses a *separate, offscreen* canvas targeting roughly 3000px wide (≈ 350 DPI for a Letter page), discarded immediately after recognition to free memory.

Before recognition, the image is preprocessed in grayscale to attack the two hardest cases—low contrast and blur:

1. **Percentile contrast stretch.** A luminance histogram is computed and the intensities between the 2nd and 98th percentiles are stretched to the full $[0, 255]$ range (ignoring outliers). Low-contrast pages are pushed toward clean black-on-white; already-punchy pages are left almost unchanged.
2. **Unsharp mask.** Edges are re-sharpened as $\text{sharp} = g + \alpha(g - g_{\text{blur}})$ with $\alpha = 1$, where g_{blur} comes from a separable box blur implemented with a sliding-window sum so its cost is $O(n)$ independent of radius.

Tesseract performs its own binarization afterward, so this stage only needs to hand it a clean, high-contrast grayscale image.

7.6 OCR Engine Orchestration

Recognition runs through a Tesseract.js *scheduler* that distributes jobs across a pool of Web Workers, so multiple pages OCR concurrently without blocking the UI. The worker count scales with CPU cores but is capped, leaving one core for rendering, because each worker loads its own copy of the language model:

$$N_{\text{workers}} = \max(1, \min(4, \text{hardwareConcurrency} - 1)). \quad (5)$$

Workers are configured for *sparse text* page segmentation (find as much text as possible, in any order), which markedly outperforms the default layout analysis on tables, captions, and scattered small text. Three language modes are supported—`eng`, `dan`, and `eng+dan` (combined)—loaded from bundled `traineddata` files. The choice is persisted in `localStorage`, and results are cached per language so switching is non-destructive.

7.7 Post-Processing for Findability

Because native find performs exact substring matching, fuzzy matching is impossible at search time. The post-processing stage instead improves the *stored* text. It drops near-zero-confidence noise, trims non-alphanumeric junk from word edges, and—critically—emits an extra *correction variant* for words that look like digit/letter confusions (e.g. `he11o` → `hello`, `ca5h` → `cash`). A token whose digits outnumber its letters (more than 50% digit characters) is left alone and treated as a genuine number or code, so correction is only applied where the OCR confusion is plausible:

```
1  const DIGIT_TO_LETTER = {
    "0": "o", "1": "l", "5": "s", "6": "b", "8": "b", "9": "g", "2": "z" };
2  // keep the raw word (selectable), then add a corrected, search-only
    variant
3  out.push({ text, bbox: w.bbox });
4  const variant = correctionVariant(text);           // null unless mixed
    letters+digits
5  if (variant && variant !== text)
6    out.push({ text: variant, bbox: w.bbox, variant: true });
```

Listing 4. Emitting a search-only correction variant for OCR tokens with likely digit/letter confusions (e.g. `he11o` → `hello`); the raw word is retained for copy/paste (`ocr-postprocess.js`).

Both the raw word and its corrected variant are injected at the same position, so a search succeeds whether the user types the true spelling or the OCR'd one. Variants are marked `variant: true` and excluded from mouse selection and copy via CSS (`user-select: none`), so copied text is never duplicated.

7.8 Normalization, Caching, and De-duplication

Recovered words are normalized to the unit square (Eq. (3)) and stored in IndexedDB, keyed by ``${docId}::${pageNum}`` where the document ID also encodes the OCR language. The store carries a *format version*; bumping it (e.g. when the preprocessing pipeline changed) transparently invalidates stale results. Empty results are cached too, so text-only pages are not re-examined on every load.

A page can carry *both* native text and image text. After OCR, words are de-duplicated against the native boxes: an OCR word is dropped only if its *centre point* falls inside a native box that already contains the same token. This centre-point test means a large OCR word whose centre lies outside a small native box is kept, erring on the side of retaining searchable text.

7.9 Search and the Custom Find Bar

A subtle rendering issue motivated replacing the native find UI: Chrome's built-in find makes the invisible text layer briefly visible in a mismatched font, which looks like oversized, doubled text over the page. FIND IN IMAGES therefore intercepts Ctrl/Cmd+F and presents its own find bar. For each query it walks the injected spans, builds a DOM `Range` around every match, and reads `getClientRects()` to draw translucent highlight overlays positioned relative to each page wrapper; the active match is styled distinctly and scrolled into view, with ↑ / ↓, Enter, and F3 navigation. Because pages become searchable progressively, an active search is re-run (debounced) as each new page finishes indexing, with the current match preserved.

8 Results and Evaluation

This evaluation is qualitative and behavioral: it characterizes what the system achieves and how its architecture behaves, rather than reporting formal accuracy or timing benchmarks, which are identified as future work in Section 9.

8.1 Functional Outcomes

The system meets its central goal: scanned and image-based PDFs that return nothing to in-page search become fully searchable in the browser. Specifically:

- **Scanned PDFs become searchable.** Pages with no text layer are OCR'd and their text injected, so queries match content that was previously invisible to search.
- **Mixed pages are handled.** Pages combining native text with text baked into images expose both, with duplicates removed so a word is never double-counted.
- **Rotated text aligns.** Because rotation is recovered from the transform matrix (Eq. (2)) rather than special-cased, highlights and selection track rotated pages and rotated runs.
- **Selection and copy work.** Injected text is real DOM text, so users can not only find but also select and copy recovered content; search-only correction variants are excluded from copy to avoid duplication.
- **Fully offline.** OCR engine, language models, and PDF.js are bundled, so no document data leaves the machine and the extension works without a network connection.

8.2 Architectural Behavior

The performance-oriented design choices produce the following observable behavior:

Table 5. Design mechanisms and the behavior they produce.

Mechanism	Observed effect
Lazy, viewport-driven rendering	Large documents open immediately; only near-viewport pages do work up front.
Idle-time background indexing	Off-screen pages still become searchable, scheduled during idle time so scrolling is never blocked.
Web Worker OCR pool	Recognition runs off the main thread on several pages at once; the interface stays responsive during OCR.
High-resolution OCR canvas + preprocessing	Small text inside images is legible to the recognizer independent of on-screen zoom.
IndexedDB caching (normalized coords)	A document is OCR'd once; re-opening or re-zooming it is instant and correct at any scale.
Correction variants	Common digit/letter misreads remain findable by their true spelling.

8.3 Component Complexity

Table 6 summarizes the relative implementation complexity of each component. The coordinate-mapping and DOM-overlay subsystems dominate: they are where most of the subtle, correctness-critical logic lives.

Table 6. Relative complexity by component.

Component	Complexity
PDF rendering (PDF.js)	Medium
OCR pipeline (Tesseract.js)	Medium
Chrome extension integration	Medium
Coordinate mapping	High
DOM text-overlay system	High

8.4 Discussion

The results validate the project’s central thesis: by converting documents into real, well-positioned DOM text, the system inherits the browser’s mature search behavior almost for free, and the engineering effort concentrates where the real difficulty lies—geometric alignment and keeping a CPU-heavy workload off the critical path. The accuracy layer (high-resolution rendering, preprocessing, and correction variants) measurably widens what is findable in practice, though it cannot fully overcome poor source scans; this is examined next.

9 Limitations and Future Work

9.1 Limitations

OCR is not perfect.

Recognition errors propagate into search. Because native find is exact, a misread character can make a word unfindable by its true spelling. Correction variants mitigate the most common digit/letter confusions but do not cover the general case.

Cost scales with the document.

Recognition is CPU-bound and runs on the client. Very large or very dense documents take correspondingly longer to fully index, even though lazy rendering keeps the document immediately readable and caching makes subsequent loads instant.

Complex layouts can misalign.

Multi-column text, unusual fonts, and heavy graphical backgrounds can produce slight overlay drift or degraded recognition.

Device-dependent performance.

Throughput depends on the number of CPU cores and the device’s overall capability, since the worker pool scales with available concurrency.

Scope is currently PDF-centric.

Image text on ordinary web pages is not yet handled; the content script that would address this is presently a placeholder.

9.2 Future Work

- **Higher OCR accuracy.** Richer post-processing—dictionary-based correction, confidence-weighted variants, and language-model re-ranking—to reduce unfindable misreads beyond the current digit/letter mapping.
- **Better rotation and skew handling.** Automatic deskew before recognition for photographed or imperfectly-scanned pages.
- **Broader language support.** Additional bundled language models and automatic script detection.
- **Acceleration.** Exploiting GPU/WebAssembly SIMD acceleration and preprocessing whole documents in the background to shorten time-to-fully- searchable on large files.
- **General web-page image text.** Extending the pipeline, via the content script, to make text inside ordinary `<img>` elements searchable, not just PDFs.
- **Quantitative evaluation.** A benchmark suite measuring recognition accuracy (e.g. character/word error rate) and per-page latency across a corpus of representative scanned documents, to turn the qualitative results of Section 8 into reproducible numbers.

10 Conclusion

FIND IN IMAGES demonstrates that the most effective way to make a scanned or image-based document searchable in the browser is not to build a search engine, but to *make the document into real text* and let the browser do what it already does well. Built as a fully client-side Manifest V3 Chrome extension, the system intercepts PDF navigations, renders pages with PDF.js, recovers missing text with in-browser Tesseract.js OCR, and injects an invisible, precisely-aligned text layer that native search, selection, and copy all operate on directly—without any server round-trip or data leaving the user’s machine.

The project’s depth lies in the parts that are easy to underestimate. Aligning recovered text with the rendered page is a four-space coordinate problem solved with an affine-transform pipeline and a measure-and-scale fitting step that stays correct under zoom, page rotation, and high-DPI rendering. Keeping a CPU-heavy workload usable is an architecture problem solved with lazy rendering, idle-time background indexing, a Web Worker pool, and resolution-independent caching. Maximizing findability is an accuracy problem attacked with high-resolution rendering, targeted image preprocessing, and search-only correction variants.

Together these turn static, unsearchable documents into searchable, web-native experiences. More broadly, the project illustrates a reusable principle for client-side engineering: bridge a new capability (OCR) into an existing, battle-tested substrate (the browser’s text and search machinery) rather than re-implementing that substrate—and invest the engineering effort precisely where the genuine difficulty lies.

References

- [1] Adobe Systems, “Document management — portable document format — part 1: PDF 1.7 (ISO 32000-1).” ISO standard, 2008.
- [2] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. Ninth International Conference on Document Analysis and Recognition (ICDAR)*, pp. 629–633, IEEE, 2007.
- [3] Tesseract.js Contributors, “Tesseract.js: Pure JavaScript OCR for more than 100 languages.” <https://tesseract.js.org/>, 2024. Accessed 2026.
- [4] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [5] Mozilla, “PDF.js: A general-purpose, web standards-based platform for parsing and rendering PDFs.” <https://mozilla.github.io/pdf.js/>, 2024. Accessed 2026.
- [6] Google, “Chrome extensions: Manifest V3 overview.” <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3>, 2024. Accessed 2026.
- [7] Google, “chrome.declarativeNetRequest api reference.” <https://developer.chrome.com/docs/extensions/reference/api/declarativeNetRequest>, 2024. Accessed 2026.
- [8] MDN Web Docs, “IndexedDB API.” https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API, 2024. Accessed 2026.
- [9] MDN Web Docs, “Intersection observer API.” https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API, 2024. Accessed 2026.